ECE 220 Computer Systems & Programming

Lecture 14 — File I/O in C

October 13, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

Running Sum, Compiled by Hand

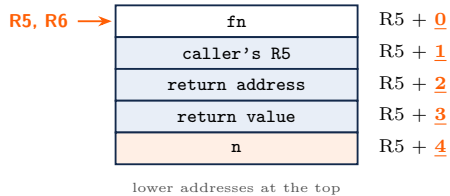
```
int RunningSum(int n) {
    int fn;
    if (n == 1)
        fn = 1;              /* base case */
    else
        fn = n + RunningSum(n - 1);
    return fn;
}
int main() {
    result = RunningSum(3);
```

`runsum.asm`, assembled at `x3000`:

```
x3000  MAIN    LD R6, USER_STACK
  ...
x3007          JSR RUNSUM
x3008          LDR R0, R6, #0
  ...
x300C  RUNSUM  ADD R6, R6, #-1
  ...
x3018          JSR RUNSUM
x3019          LDR R2, R6, #0
```

A recursive function calls itself before it returns, so several calls are alive at once: `RunningSum(3)` has **3** above `main`. Each call needs its own activation record (AR) on the run-time stack (RTS), and one subroutine, `RUNSUM`, builds them all.

The AR of one call, `RunningSum(n)`



`n` is read with `LDR R1, R5, #4`; `fn` is written with `STR R1, R5, #0`.

The RTS at the Base Case

		contents	what it is
R5, R6 →	xFDEF	1	RunningSum(1): fn
	xFDF0	xFDF4	caller's R5
	xFDF1	x3019	return address
	xFDF2	1	return value
	xFDF3	1	arg n
	xFDF4		RunningSum(2): fn
	xFDF5	xFDF9	caller's R5
	xFDF6	x3019	return address
	xFDF7		return value
	xFDF8	2	arg n
	xFDF9		RunningSum(3): fn
	xFDFA	xFDFE	caller's R5
	xFDFB	x3008	return address
	xFDFC		return value
	xFDFD	3	main pushed n = 3
	xFDFE		main: result

`RunningSum(1)` just wrote its return value. Fill in the RTS; mark R5 and R6.

```
x3000  MAIN    LD R6, USER_STACK
  ...
x3007          JSR RUNSUM
x3008          LDR R0, R6, #0
  ...
x300C  RUNSUM  ADD R6, R6, #-1
  ...
x3018          JSR RUNSUM
x3019          LDR R2, R6, #0
```

A `JSR` saves the address of the instruction after it: `main`'s `JSR` at x3007 makes `RunningSum(3)` return to **x3008**; the `JSR` inside `RUNSUM` at x3018 makes each recursive call return to **x3019**.

The calls return, in order, **1**, **3**, **6**; ARs on the RTS right now: **4**.

In-Class Problem: RUNSUM, Part 1 — Build the AR

`main` calls `RunningSum(3)` (`runsum.asm`, lines 4–17): push 3, `JSR RUNSUM`, read the result at `M[R6]`, pop two words.

- **Registers**: R5 frame pointer, R6 stack pointer, R1 `n`, R2 `n - 1` and the returned value
- `main` is a caller: set-up before the `JSR`, tear-down after it. `RUNSUM` is the callee, and inside its body a caller as well
- The AR is built before `n` is tested, so the base case also has an AR to tear down
- Seven instructions build the AR; three test `n`

```
; ---- callee set-up ----
RUNSUM  ADD  R6, R6, #-1      ; return-value slot
        ADD  R6, R6, #-1
        STR  R7, R6, #0       ; push return address
        ADD  R6, R6, #-1
        STR  R5, R6, #0       ; push caller's R5
        ADD  R5, R6, #-1      ; R5 -> first local (fn)
        ADD  R6, R6, #-1      ; local: fn
; ---- if (n == 1) ----
        LDR  R1, R5, #4       ; R1 <- n
        ADD  R2, R1, #-1      ; R2 <- n - 1
        BRz  BASE             ; n == 1
```

`ADD R2, R1, #-1` sets the condition codes from `n - 1`, so `BRz` tests `n == 1` and R2 already holds the next argument.

In-Class Problem: RUNSUM, Part 2 — the Recursive Call

`fn = n + RunningSum(n - 1);` Inside `RUNSUM`, the subroutine is now the caller.

- Caller set-up: push **n - 1**, then `JSR RUNSUM`
- When the `JSR` comes back, the return value is at **M[R6]**
- Caller tear-down pops **2** words
- `n` is loaded from `R5 + 4` a second time because **the call changed R1**
- The sum goes to `fn`: `STR R1, R5, #0`

Registers are not saved across a call. Everything `RUNSUM` needs after the `JSR` comes from its own AR: `n` at R5+4, `fn` at R5.

```
; ---- recursive case ----
        ADD  R6, R6, #-1
        STR  R2, R6, #0        ; push n - 1
        JSR  RUNSUM            ; RunningSum(n - 1)
        LDR  R2, R6, #0        ; R2 <- its return value
        ADD  R6, R6, #2        ; pop RV and argument
        LDR  R1, R5, #4        ; n again: R1 was changed
        ADD  R1, R1, R2        ; n + RunningSum(n - 1)
        STR  R1, R5, #0        ; fn = the sum
        BR   RETURN
```

In-Class Problem: RUNSUM, Part 3 — Return and Tear Down

- BASE: `fn` = 1
- `RETURN` copies `fn` into `R5` + 3, the return-value slot
- Tear-down pops the local, then restores R5 and R7
- After `RET`, R6 points at the return value
- Run it: `result` = 6, stored at xFDFE

The same six instructions tear down every AR: `ADD R6, R5, #1` finds the bookkeeping words from R5.

One subroutine, one convention, any depth of recursion. The C compiler does exactly this.

```
; ---- base case ----
BASE    AND  R1, R1, #0
        ADD  R1, R1, #1
        STR  R1, R5, #0        ; fn = 1
; ---- return fn ----
RETURN  LDR  R1, R5, #0        ; R1 <- fn
        STR  R1, R5, #3        ; return value <- fn
; ---- callee tear-down ----
        ADD  R6, R5, #1        ; pop the local
        LDR  R5, R6, #0        ; restore caller's R5
        ADD  R6, R6, #1
        LDR  R7, R6, #0        ; restore return address
        ADD  R6, R6, #1        ; R6 -> return value
        RET
```

A First File Program: intro.c

```c
/* Write a self-introduction to intro.txt */
#include <stdio.h>

int main() {
    FILE *file;
    char buffer[100];

    file = fopen("intro.txt", "w");
    if (file == NULL) {
        printf("Cannot open intro.txt\n");
        return -1;
    }
    printf("Introduce yourself: ");
    fgets(buffer, 100, stdin);    /* keyboard */

    fputs("Your self introduction: ", file);
    fputs(buffer, file);          /* file     */
    fclose(file);

    fputs(buffer, stdout);        /* monitor  */
    return 0;
}
```

A First File Program: `intro.c`

```c
/* Write a self-introduction to intro.txt */
#include <stdio.h>

int main() {
    FILE *file;
    char buffer[100];

    file = fopen("intro.txt", "w");
    if (file == NULL) {
        printf("Cannot open intro.txt\n");
        return -1;
    }
    printf("Introduce yourself: ");
    fgets(buffer, 100, stdin);    /* keyboard */

    fputs("Your self introduction: ", file);
    fputs(buffer, file);          /* file     */
    fclose(file);

    fputs(buffer, stdout);        /* monitor  */
    return 0;
}
```

On the monitor

```
Introduce yourself:  I study ECE.
I study ECE.
```

In the file `intro.txt`

```
Your self introduction:  I study ECE.
```

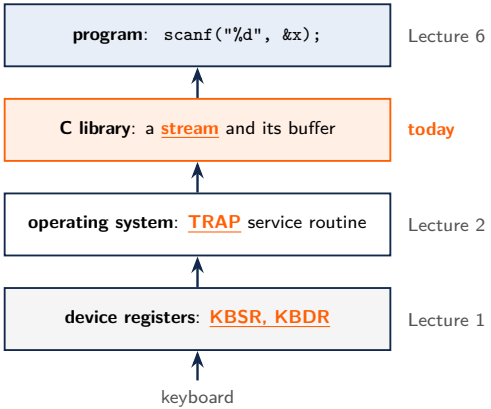
By the end of today you will know what a `FILE *` is, and what lies between each of these calls and the keyboard, the monitor, and the disk.

Today's Three Questions

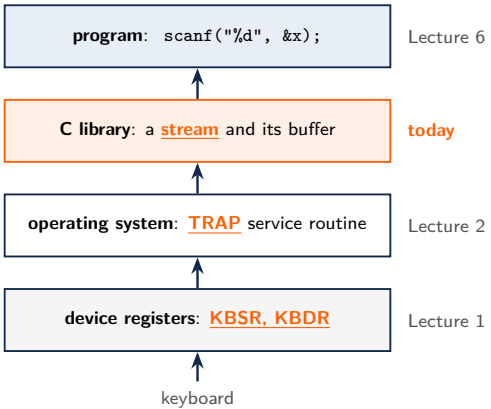
In Lecture 1 our program polled KBSR in a loop.
`intro.c` reads the keyboard and names no device register.

1. What is a `FILE *`, and why does the same `fputs` write to a file and to the monitor?
2. What lies between the program and the device?
3. How does the program learn that a call failed, or that the input ran out?

From Device Registers to Streams



From Device Registers to Streams



Lecture 1: our program waited for the ready bit.

```
POLL  LDI  R1, KBSR_ADDR  ; R1 <- KBSR
      BRzp POLL           ; not ready: wait
      LDI  R0, KBDR_ADDR  ; R0 <- KBDR
```

Lecture 2: the polling loop moved into the OS. Our program calls `GETC`.

In C: the device works by an I/O protocol, such as memory-mapped I/O. The program makes one function call.

Each layer hides the one below it. The layer the C library adds is the stream.

One Abstraction: the Stream

A **stream** is a sequence of ASCII characters, processed in the order in which they were added.

`intro.c` uses three streams:

```
fgets(buffer, 100, stdin);    /* keyboard */
fputs(buffer, file);          /* file     */
fputs(buffer, stdout);        /* monitor  */
```

The characters the user types, the characters in a file, the characters printed on the monitor: each is one stream.

`printf` and `scanf` have the stream built in:

`printf(...)` is `fprintf(`**stdout**`, ...)`

`scanf(...)` is `fscanf(`**stdin**`, ...)`

Three standard streams

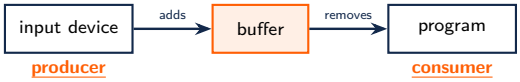
`stdin`	input	keyboard
`stdout`	output	monitor
`stderr`	error messages	**monitor**

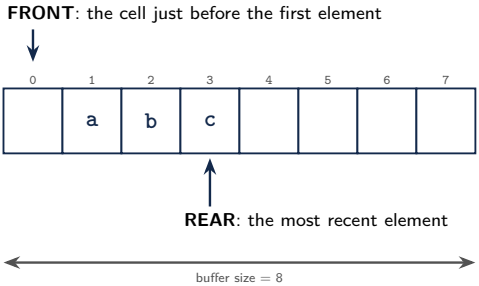
All three are open when `main` starts. Their type, and the type of `file` in `intro.c`: **FILE ***

What is a `FILE`?

- A type from `<stdio.h>`. One `FILE` holds what the library knows about one open stream: the file or device, the position in it, the buffer.
- It is a `struct`, the topic of Lecture 15. The program stores the pointer and passes it to every call.

Stream Buffering





- The buffer lets the device and the program run independently, each at its own speed
- A video player does the same: it plays from its buffer while the network stalls
- The producer adds at **REAR**; the consumer removes at **FRONT**
- First in, first out: a **queue**
- **Circular**: after cell 7 comes cell **0**
- Empty when **FRONT == REAR**
- Full when **(REAR + 1) % 8 == FRONT**

One cell stays unused, so a full buffer and an empty buffer give different tests.

In the Buffer: y, Enter

```
int main() {
    char inChar1, inChar2;

    printf("ECE student? (y/n): ");
    inChar1 = getchar();
    printf("Taking ECE 220? (y/n): ");
    inChar2 = getchar();

    printf("\ninChar1 = %d\n", inChar1);
    printf("inChar2 = %d\n", inChar2);
    return 0;
}
```

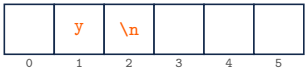
The keyboard's characters reach the program when the user presses **Enter**.

The program prints `inChar1` = **121, 'y'**

and `inChar2` = **10, the newline**

The user types `y`, then Enter. Draw the buffer:

after `y` and Enter



FRONT = 0
REAR = 2

after the first `getchar`



FRONT = 1
REAR = 2

after the second `getchar`



FRONT = 2
REAR = 2

The second `getchar` finds the newline in the buffer and returns at once. One more `getchar()` after the first removes it.

Opening and Closing a File

`FILE *fopen(char *filename, char *mode)`

success: a pointer to the stream failure: **NULL**

`"r"` read; the file must exist

`"w"` write; an existing file is **erased**

`"a"` append; writing starts at the **end**

`"r+"`, `"w+"`, `"a+"`: read and write

`int fclose(FILE *stream)`

writes out what is left in the buffer, then releases the file

success: **0** failure: `EOF`, a macro in `<stdio.h>`, commonly -1

Opening and Closing a File

`FILE *fopen(char *filename, char *mode)`

success: a pointer to the stream failure: **NULL**

`"r"` read; the file must exist

`"w"` write; an existing file is **erased**

`"a"` append; writing starts at the **end**

`"r+"`, `"w+"`, `"a+"`: read and write

`int fclose(FILE *stream)`

writes out what is left in the buffer, then releases the file

success: **0** failure: `EOF`, a macro in `<stdio.h>`, commonly -1

```
file = fopen("intro.txt", "w");
if (file == NULL) {
    printf("Cannot open intro.txt\n");
    return -1;
}
fclose(file);
```

In-class

- `intro.c` runs in a folder where it may not create a file. It prints **Cannot open intro.txt**
- A program calls `fputs`, then crashes before `fclose`. The file holds **nothing**

The characters were still in the buffer.

What Each Call Returns

Call	What it does	On success returns	On failure
`fprintf(stream, format, ...)`	writes formatted text	number of characters written	a negative number
`fscanf(stream, format, ...)`	reads formatted items	number of items read	EOF
`fgetc(stream)`	reads one character	the character, as an `int`	EOF
`fputc(ch, stream)`	writes one character	the character written	EOF
`fgets(string, num, stream)`	reads a line	`string`	NULL
`fputs(string, stream)`	writes a string	a non-negative number	EOF
`feof(stream)`	end of file reached?	nonzero at the end of the file, 0 before it	

`getchar()` is `fgetc(stdin)`, and `putchar(ch)` is `fputc(ch, stdout)`.

In-class

- The file holds `2 x.` `fscanf(in, "%d %d", &m, &n)` returns **1**
- The file is empty. The same call returns **EOF**
- Why does `fgetc` return an `int`? `EOF` must differ from every character.

Reading Until the End of a File

```
in_file = fopen("filecopy.c", "r");
if (in_file == NULL) {
    printf("Unable to open filecopy.c\n");
    return -1;
}
out_file = fopen("filecopy_out.c", "w");
if (out_file == NULL) {
    printf("Unable to open filecopy_out.c\n");
    fclose(in_file);
    return -1;
}
fprintf(out_file, "/* a copy of filecopy.c */\n");
while (fgets(str, 32, in_file) != NULL)
    fputs(str, out_file);

fclose(in_file);
fclose(out_file);
```

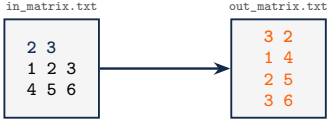
`filecopy.c` copies its own source file, one piece at a time.

- `fgets(str, 32, in_file)` stores at most **31** characters, then **`'\0'`**
- A line of 50 characters arrives in **2** calls
- `fgets` returns `NULL` when **no characters are left**
- Why does the second failure branch call `fclose(in_file)`?
  `in_file` is open. Every `fopen` that succeeds gets one `fclose`.

`feof` turns nonzero only after a read has reached the end, so the loop tests what `fgets` returned.

In-Class Problem: Transpose a Matrix from a File

Read an $m \times n$ matrix from `in_matrix.txt`; write its transpose to `out_matrix.txt` with `fscanf` and `fprintf`. Line 1 of each file holds the dimensions.




```c
#define MAX 10    /* largest m or n accepted */
int main() {
    FILE *in_file, *out_file;
    int in_matrix[MAX][MAX];
    int m, n, r, c;
    /* open in_matrix.txt for reading */
    in_file = fopen("in_matrix.txt", "r");
    if (in_file == NULL)
        return -1;
```

```c
    /* read m and n; check that they fit */
    if (fscanf(in_file, "%d %d", &m, &n) != 2
        || m < 1 || m > MAX || n < 1 || n > MAX) {
        fclose(in_file);
        return -1;
    }
    /* read the m x n elements, row by row */
    for (r = 0; r < m; r++)
        for (c = 0; c < n; c++)
            fscanf(in_file, "%d", &in_matrix[r][c]);
    /* close in_matrix.txt */
    fclose(in_file);
```

`fscanf` returns 2 only when both `m` and `n` were read. When the check fails, `in_file` is open, so the program closes it before it returns.

In-Class Problem: Write the Transpose

```
    /* open out_matrix.txt for writing */
    out_file = fopen("out_matrix.txt", "w");
    if (out_file == NULL)
        return -1;
    /* write the dimensions of the transpose */
    fprintf(out_file, "%d %d\n", n, m);
    /* write the elements of the transpose */
    for (c = 0; c < n; c++) {
        for (r = 0; r < m; r++)
            fprintf(out_file, "%d ", in_matrix[r][c]);
        fprintf(out_file, "\n");
    }
    /* close out_matrix.txt */
    fclose(out_file);
    return 0;
}
```

- The transpose has **n** rows and **m** columns
- Output row `c` holds input **column c**
- Outer loop: **c** inner loop: **r**
- Lecture 11's `transpose` filled a second array. Here each element goes to the file, and one array is enough.
- `in_matrix.txt` starts with `20 30`. The program **returns -1, writes no file**

MAX wastes memory on a 2×3 matrix and refuses a 20×30 one. Lecture 16: ask for $m \times n$ integers at run time.

Summary & What's Next

Today

- **Recursion in LC-3**: one AR on the RTS per live call, built and torn down by the same code; after a `JSR`, every value comes back from the AR, not from a register
- **Streams**: a sequence of characters, read or written in order; `stdin`, `stdout`, `stderr`, and every file a program opens are each a `FILE *`
- **Buffers**: a circular queue between the producer and the consumer, with FRONT and REAR; keyboard input reaches the program at Enter
- **File functions**: `fopen` with a mode, then `fprintf`/`fscanf`, `fgetc`/`fputc`, `fgets`/`fputs`, then `fclose`; test what each call returns: `NULL`, `EOF`, or a count
- **Transpose from a file**: read and check `m` and `n`, read row by row, write column by column

Next lecture

- Data structures: `enum`, `struct`, `union`

Code from today: `running.c`, `runsum.asm` and `runsum_skel.asm`, `intro.c`, `kb_poll.asm`, `in_buffer.c`, `filecopy.c`, `ftranspose.c` with its input `in_matrix.txt`.